

CineSeat: Code Review Process & Debugging Strategy

Week 5 Internship

1. Code Review Process

Purpose: Establish systematic peer review practices to catch defects early, improve code quality, and share knowledge across the team.

1.1 Code Review Workflow

Code review is mandatory for all commits to the main and develop branches. The workflow follows these stages:

Stage	Description
1. Developer Submits	Developer creates a Pull Request (PR) with linked JIRA ticket and descriptive title. Example: "SEAT-042: Fix race condition in ReentrantLock during concurrent reservation"
2. Automated Checks	CI/CD pipeline runs: JUnit 5 tests, SonarQube analysis, code coverage (target ≥80%), checkstyle validation (Google Java Style). PR cannot proceed if tests fail.
3. Peer Review	At least 2 senior developers review code within 24 hours. Checkers focus on logic, concurrency, Redis interactions, and Spring Boot configuration.
4. Feedback & Iteration	Reviewer leaves comments on specific lines. Developer addresses concerns or justifies decisions. All conversations resolved before approval.
5. Approval & Merge	Once 2 approvals received and CI passes, developer squashes commits and merges to target branch.

1.2 Code Review Checklist by Component

Different modules require focus on specific concerns:

Component	Key Review Focus	Example Red Flags
-----------	------------------	-------------------

SeatReservationService (Seat Locking)	• ReentrantLock acquisition & release • Timeout handling on lock fails • No nested locks causing deadlock • Redis Redisson lock fallback logic	• Lock held without try-finally • Missing timeout on lockInterruptibly() • Synchronous Redis calls in main thread
SeatHoldManager (Redis TTL)	• ZSET operations (zadd, zrem) • TTL expiration logic (30sec → payment) • Score-based ranking for VIP priority • Cache invalidation on hold release	• Direct set() instead of expire() • Missing WATCH for transaction isolation • Off-by-one errors in ZSET queries
PaymentProcessor (Idempotency)	• Idempotency key generation & storage • Duplicate payment prevention • Transaction rollback on failure • External API retry with backoff	• No idempotency key checking • Transaction left open on exception • Immediate retry without jitter
KafkaEventPublisher (Event Ordering)	• Topic partitioning strategy (by theatreId?) • Event serialization (JSON schema valid?) • Retry policy & dead-letter queue • Consumer offset management	• Random partition (breaks ordering) • Unversioned event schema • Lost messages on connection failure

1.3 Best Practices & Collaboration Tools

Actionable Feedback: Instead of "This is wrong", say: "Race condition: ReentrantLock may be released by wrong thread if exception thrown. Wrap in try-finally at line 45."

Positive Tone: Frame concerns as questions: "Have we considered if another thread might bypass this check?" vs "You forgot to synchronize."

Timely Reviews: Review within 24 hours; large PRs (>400 LOC) should be split to speed feedback. Mark PRs as WIP if still drafting.

Tooling: GitHub (PR comments), SonarQube (code smells), JaCoCo (coverage gaps), and Slack notifications for review requests.

Review Limits & Cognitive Fatigue: Limit PR reviews to a maximum of 400 LOC per hour, and cap review sessions at 60 minutes to prevent fatigue-induced omissions.

OWASP Security Checklist: Verify that payment tokens and secrets are encrypted at rest and in transit, and ensure that proper security authorizations are validated on endpoints modifying seat holds.

2. Debugging Methodology

Purpose: Provide structured, systematic approaches to identify, isolate, and resolve defects in concurrent, distributed systems.

2.1 Debugging Methodology Stages

Stage	Activities	CineSeat Example
Issue Identification	• Monitor application logs & error rates • Analyze exception stack traces • Cross-check with user reports • Review metrics (seat hold failures, payment timeouts)	Payment timeout: User X unable to complete reservation. Error logs show: "Redis ZSET add timed out after 5s. theatreId=101, showId=5."
Hypothesis Formation	• Root cause analysis (RCA) meeting • Review recent commits to affected module • Examine state of Redis / database at time of failure • Check thread dumps & lock contention	Hypothesis: High concurrency on showId=5 (popular movie) + weak Redis connection pooling caused ZSET operations to queue, leading to timeout.
Isolation & Reproduction	• Set up isolated test environment (clone prod data) • Write minimal reproduction test (JUnit + Mockito) • Use breakpoints & step through code • Enable debug logging (log at TRACE level around problem area)	Test: Mock 1000 concurrent seat holds for showId=5, measure Redis op latency. Found: connection pool exhausted at ~800 concurrent ops.
Resolution & Verification	• Implement fix (increase pool size, add circuit breaker) • Write unit & integration tests to prevent regression • Deploy to staging, monitor metrics • Verify fix in production after deployment	Fix: Increased Redis connection pool from 10 to 50 conns. Added HystrixCommand with fallback. Verified: 5000 concurrent ops now ≤100ms p99.

2.2 Tools & Techniques by Scenario

Different problems require different debugging approaches:

Scenario	Tools & Techniques	Configuration
Concurrent Lock Deadlock	• jstack (thread dump) • VisualVM (thread state)	jstack \$PID grep -i blocked redisson.setLoggerFactory(Slf

	analysis) - Redisson DEBUG log - Write test with 100+ threads	4jLoggerFactory.INSTANCE) Log: org.redisson.client at DEBUG level
Redis Timeout / Slow Ops	- redis-cli MONITOR (live command stream) - redis-cli INFO stats (throughput) - Micrometer Prometheus metrics - WireShark (network packet trace)	docker exec redis redis-cli MONITOR Spring: management.endpoints.web. exposure.include=prometheu s Metric: redis.commands.duration
Payment Idempotency Bug	- Mockito mocks for external payment API - Database query (check transaction_id in payments table) - JUnit parametrized test (duplicate calls) - Spring Boot actuator /health endpoint	@Test void testDuplicatePaymentBlocke d() throws... Payload: {idempotency_key: "abc123", amount: 500}
Kafka Consumer Lag	- Kafka consumer-groups CLI - Prometheus metrics (lag_sum by partition) - Log pattern: "Message lag exceeds SLA" - Replay topic with old offset	kafka-consumer-groups -- bootstrap-server localhost:9092 --group seat- reservation --describe lag > 10000 indicates backlog
Spring Boot Config Binding	- Property file validation at startup - ConfigurationProperties with @Validated - Spring Boot Actuator /env (sanitized) - Maven spring-boot-maven-plugin goal verify-plugin-configuration	@ConfigurationProperties(pr efix="app.redis") @Validated public class RedisConfig { @NotBlank String host; }

2.3 Structured Logging Strategy

All logs follow a structured JSON format for easy parsing and correlation:

Level	Use Case	CineSeat Example
ERROR	Exceptions, failed transactions, unrecoverable states	{"level":"ERROR","timestamp":"2025-01-15T10:30:45Z","event":"payment_failed","userId":"user_42","orderId":"ord_789","amount":500,"error":"Payment

		GatewayException: timeout after 30s","trace":"..."}}
WARN	Recoverable issues, SLA breaches, retries	{"level":"WARN","timestamp":"2025-01-15T10:29:50Z","event":"seat_hold_near_expiry","theatreId":101,"holdId":"hold_555","remainingMs":5000,"status":"payment_pending"}
INFO	Major flow milestones, user actions	{"level":"INFO","timestamp":"2025-01-15T10:28:30Z","event":"seat_reserved","userId":"user_42","theatreId":101,"seats":["A1","A2"],"holdDuration":"10m"}
DEBUG	Lock acquisitions, Redis ops, state transitions	{"level":"DEBUG","timestamp":"2025-01-15T10:28:31Z","event":"redisson_lock_acquired","lockKey":"seat:101:A1","waitMs":5,"holdMs":10000}

Logback Configuration (logback-spring.xml):

```
<appender name="JSON"
class="ch.qos.logback.core.FileAppender">
  <file>logs/cinseat.json</file>
  <encoder
class="net.logstash.logback.encoder.LogstashEncoder"/>
</appender>
<root level="INFO">
  <appender-ref ref="JSON"/>
</root>
```

2.4 Distributed Tracing & APM Integration

Utilize OpenTelemetry and Jaeger for end-to-end trace propagation across SeatReservationService, PaymentProcessor, and KafkaEventPublisher. Trace IDs are injected into HTTP headers and Kafka event metadata to track asynchronous event flows and payment lifecycles.

2.5 Memory & Resource Leak Analysis

Capture JVM heap dumps using jcmd or analyze them with Eclipse MAT to identify memory leaks. Monitor thread leaks and unreleased Redisson locks by tracking active thread counts and connection pool metrics.

3. Documentation & Defect Management

Purpose: Maintain complete records of all issues discovered, investigated, and resolved for long-term trend analysis and root-cause prevention.

3.1 Defect Classification & Tracking

All defects are logged in JIRA with the following fields:

Field	Description & CineSeat Example
Summary	Brief description. Example: "SEAT-156: Race condition in SeatHoldManager.releaseExpiredHolds causing double-release"
Type	Bug, Feature, Technical Debt. Example: Bug (concurrency flaw)
Severity	Critical (payment fails), High (seat holds broken), Medium (slow query), Low (UI glitch). Example: Critical
Priority	P0 (deploy fix today), P1 (this sprint), P2 (next sprint), P3 (backlog). Example: P0
Component	SeatReservationService, SeatHoldManager, PaymentProcessor, KafkaEventPublisher. Example: SeatHoldManager
Root Cause	Analysis of why the bug occurred. Example: "Missing synchronization on ZSET removal. Concurrent threads both call ZREM on same holdId, one succeeds, other fails silently."
Steps to Reproduce	Minimal repro. Example: "1. Mock 500 concurrent seat releases. 2. Assert each holdId removed exactly once. 3. Observe: one remove succeeds, others log race."

3.2 Defect Lifecycle & SLA

Defects move through states with defined SLAs:

State	SLA	Owner & Action
New	Triage within 24h	QA lead assigns to developer, adds initial analysis
In Progress	Fix within 2–5 days (P0: 1 day)	Developer investigates, writes unit test, codes fix
Testing	QA test within 1 day	QA runs full test suite + regression tests on staging
Resolved	Code review within 1 day	PR reviewed, merged to develop, awaiting release
Closed	Verify in production	Fix deployed; monitoring confirms issue resolved

		Regression-Test Mandate Every bug fix must be accompanied by an automated JUnit or Integration test that explicitly fails before the fix is applied and passes after verification.
--	--	--

3.3 Quality Metrics & Reporting

Track these metrics weekly in team dashboards:

Defect Density: Defects per 1000 LOC. Target: < 2/1000. If > 3, trigger code review refresh or training.

Mean Time to Resolution (MTTR): Average days from New to Closed. Target: P0 = 1 day, P1 = 3 days, P2 = 7 days.

Escape Rate: Defects found in production / total defects. Target: < 5%. High escapes indicate QA gaps.

Code Coverage: Target $\geq 80\%$ for core modules (SeatReservationService, PaymentProcessor). Track via JaCoCo.

Test Case Pass Rate: Should be 100%. If < 95%, investigate flaky tests and environment issues.

3.4 Standardized Root Cause Analysis (RCA) Template

All major incidents require an RCA document following this template:

RCA Section	Description & Details
Incident Timeline	Chronological log of events from detection to mitigation.
Five Whys	Iterative interrogation method to drill down to the fundamental root cause.
Action Items	Categorized list of Preventative vs. Corrective actions.
Owner	Designated individual responsible for tracking action item execution.

4. Remote Collaboration Techniques

Purpose: Maintain effective debugging and review workflows in distributed teams across time zones.

Technique	Implementation for CineSeat
Async Code Review	GitHub PRs with clear description, thread-safe code annotations, and inline explanations. Avoid synchronous meetings; use comments. Review within 24h SLA.
Debugging Pairing	Use VS Code Live Share for pair debugging. Set up screen recording (OBS) for async replay. Document findings in Confluence for future reference.
Slack Incident Channel	Create #cineseat-incidents for P0/P1 bugs. Daily standup post at 9 AM UTC with: issue summary, current hypothesis, blockers. Ping owners for updates.
Knowledge Sharing	Weekly 30min "Debugging Deep Dive" where team shares interesting issues & resolutions. Rotate facilitators. Record for timezone-spread team members. Blameless Post-Mortem Principles: Prioritize system and process improvements over assigning individual blame during incident reviews. Incident Response Roles: Define clear operational roles during production outages: Incident Commander (leads coordination), Lead Investigator (drives technical diagnosis), and Communications Lead (manages stakeholder updates).

5. Case Studies: Real CineSeat Debugging Scenarios

Case Study 1: Double-Release Race Condition in Seat Holds

Scenario: Users report that reserved seats suddenly become available mid-payment, causing payments to fail with "Seat no longer available" error.

Phase	Details
Identification	Alert: Payment timeout rate jumped from 0.5% to 8% on 2025-01-14. Logs show duplicate "Hold released" messages for same holdId within 100ms window.
RCA	SeatHoldManager.releaseOnExpiry() called via: (a) TTL listener on Redis ZSET, (b) Manual PaymentFailureHandler. Both call ZREM without checking if already removed. Second caller gets silent success, but no hold exists.
Reproduction	JUnit test: 1. Add hold to ZSET. 2. Trigger TTL expire + manual release simultaneously via CountdownLatch. 3. Assert second ZREM returns 0 (deleted 0 records), but code treats any ZREM response as success.
Fix Applied	Check ZREM return value. If 0, skip downstream cleanup. Code: <pre>if (redisson.getZSet("holds").remove(holdId) == 0) { logger.debug("Hold already released, skipping cleanup"); return; }</pre>
Verification	Stress test: 10,000 concurrent hold releases. Assert each holdId removed exactly once. Monitoring: Payment timeout rate dropped to 0.3% within 1 hour of deploy.

Case Study 2: Redis Connection Pool Exhaustion Under Load

Scenario: Every Friday evening (peak booking time), seat reservations freeze for 30–60 seconds. Users complain about timeouts; support team reports 1000+ failed transactions.

Phase	Details
Identification	Friday 6 PM: error logs flood with "RedisConnectionException: Connection pool exhausted (10/10 active)". Response latency p99 = 45s. Metric: redis.commands.wait_time_ms spikes to 40,000ms.
Investigation	redis-cli INFO stats shows throughput = 800 commands/sec, but pool configured for max

	10 connections. At ~800 ops/sec ÷ 10 = 80ms/op, timeout of 5s should allow buffering, but pool waiting queue grows faster than ops complete.
Hypothesis	Each seat hold creates 3 Redis ops: (1) ZADD to holds ZSET, (2) SET hold metadata, (3) EXPIRE. With 800/sec throughput and 3 ops each, = 2400 total ops/sec. Pool drains before any connection released back.
Fix Applied	Increased Redisson connection pool: Config.setConnectionPoolSize(50) // was 10 Config.setMinIdleConnections(10) // prealloc Also added HystrixCommand with 10s timeout and fallback (degrade to synchronous lock).
Verification	Load test: 5000 concurrent seat holds. p99 latency = 85ms (was 45s). Support reports: 0 timeout errors Friday next week. Monitoring dashboard shows pool utilization at steady 20/50.

Case Study 3: Duplicate Payment Charges (Idempotency Bug)

Scenario: One user charged twice for same booking. Payment service returned 200 OK, but network timeout caused client to retry. Second charge posted to bank.

Phase	Details
Identification	User reports: bank shows 2 × \$25 charges, but CineSeat booking shows 1 reservation. Invoice sent with only 1 charge. Logs show 2 PaymentService.processPayment() calls with same orderId within 5 seconds.
RCA	PaymentProcessor did NOT check for existing idempotency key before calling external payment gateway. First call succeeded (charge posted). Network timeout. Client auto-retry. Second call treated as new transaction, charged again. No deduplication.
Reproduction Test	<pre>@ParameterizedTest @ValueSource(ints = {1, 2, 3}) // simulate retries void testDuplicatePaymentIsIdempotent(int attempt) { String idempotencyKey = "ord_789_v1"; PaymentResponse r1 = paymentProcessor.processPayment(orderId,</pre>

	<pre> idempotencyKey, 2500); PaymentResponse r2 = paymentProcessor.processPayment(orderId, idempotencyKey, 2500); assertEquals(r1.transactionId, r2.transactionId); } </pre>
Fix Applied	Before calling external API: 1. Check DB: SELECT * FROM payment_ledger WHERE idempotency_key = ? 2. If found, return cached response (do not charge again). 3. If not found, call API, store response with idempotency_key in transaction.

To scale this system-wide and reduce code duplication, a reusable custom @Idempotent AOP annotation was implemented across all transactional payment methods.	
Verification	Integration test with TestContainers (PostgreSQL). Inject payment API mock that returns success on first call, timeout on retry. Assert: single charge, single DB record, idempotency maintained.

Case Study 4: Kafka Consumer Lag & Out-of-Order Events

Scenario: Seat "Release" event processed before "Reserved" event, leaving seat in invalid state. Notifications sent out of sequence (e.g., "Thank you for your booking" after "Refund issued").

Phase	Details
Identification	Audit log shows: Event_1 (SeatReserved, ts=1000ms), Event_2 (SeatReleased, ts=2000ms), but consumer processed Event_2 first. Seat state = "released" when Event_1 arrives (rejected as invalid). Notification order: Refund email, then Confirmation email.
RCA	Kafka producer publishes to random partition. No key specified. Seat A1 events go to partitions: Reserved → partition 0, Released → partition 2. Consumers read independently; partition 2 catches up first.

Hypothesis	Need to partition by key (theatreId:seatId) to ensure ordering. All events for a seat route to same partition, processed in order.
Fix Applied	KafkaProducer config: producer.send(new ProducerRecord<>("seat-events", "key=" + theatreId + ":" + seatId, eventJson)); Now all events for a seat go to same partition (partitionId = hash(key) % partitionCount), maintaining FIFO order.
Verification	Integration test: produce 100 events for same (theatreId, seatId) pair in rapid sequence. Assert consumer receives in exact order. Production: monitoring shows event lag = 0 (no backlog), notification order correct.

6. Summary & Key Takeaways

1. Code reviews are not gate-keeping—they are knowledge-sharing and defect-prevention investments.
2. Structured debugging (identify → hypothesize → reproduce → verify) reduces MTTR by 50%.
3. Distributed systems (Redis, Kafka, async payments) require deeper monitoring and testing.
4. Idempotency, ordering, and concurrency are non-functional requirements—test them explicitly.
5. Remote teams thrive with async-first processes (recorded sessions, detailed PR comments, Slack updates).